

Confidence-Aware Guardrailed Agentic RAG for IoT Intrusion Mitigation Planning

[Open reproducibility package](#)

Joy Dutta, Hossien B. Eldeeb, Samara Mayhoub, Ali Ismail Awad, Ezedin Barka, and Hassnaa Moustafa
Artifact version 1.1.0 | 10 September 2026

A self-contained handover from CICIoT2023 traffic observations to source-traceable, deterministically bounded mitigation intent. The prototype is offline and does not execute network actions.

Contents

1. Purpose	3
2. The Main Idea	3
3. Repository Requirements	3
4. Installation	3
5. Dataset Acquisition	4
6. How CICIoT2023 Is Used	4
7. Split And Leakage Controls	4
8. Detector And Calibration	5
9. Confidence Routing	5
10. The Fixed 160-Alert Sample	5
11. Standards Retrieval	5
12. Planning And Guardrails	5
13. Action-Level Evidence Gate	6
14. Independent Model Reviews	6
15. Cost Controls	7
16. Exact Execution Order	7
17. Where To Find The Evidence	7
18. Verification	8
19. Responsible Interpretation	8
20. Troubleshooting	8

1. Purpose

This guide explains how to inspect or repeat the full workflow, from CICIoT2023 traffic records to a bounded mitigation intent. It is written for readers who may be new to intrusion detection, confidence calibration, RAG, or deterministic guardrails.

The prototype is an offline decision-support experiment. It does not send commands to a gateway or device. Every final plan has status PROPOSED_NOT_EXECUTED.

2. The Main Idea

The workflow combines several components because no single component answers the whole safety question.

- The detector estimates the likely attack family.
- Calibration makes the confidence score easier to interpret.
- Confidence routing decides whether an alert can continue to planning or should receive more evidence and review.
- Retrieval gives the planner relevant official security text.
- The model proposes a compact mitigation plan.
- Deterministic guardrails enforce format, scope, parameter, approval, and action-policy rules.
- The evidence gate checks every action separately and removes weak recommendations.

The final object is a reviewable proposal, not an executed network change.

3. Repository Requirements

- Python 3.12
- At least 20 GB of free disk space for full dataset extraction and generated artifacts
- The registered CICIoT2023 CSV archive for detector reproduction
- An OpenAI API key only for fresh planning and evidence-checker calls
- No Docker requirement

The saved result records, reports, tests, and documentation can be inspected without the dataset or an API key.

4. Installation

```
[bash]
git clone
https://github.com/joy-dutta/Guardrailed-Agent-ic-RAG-for-IoT-Intrusion-Mitigation-Planning.git
cd Guardrailed-Agent-ic-RAG-for-IoT-Intrusion-Mitigation-Planning
python -m venv .venv
```

PowerShell:

```
[powershell]
.\.venv\Scripts\python.exe -m pip install --upgrade pip setuptools
.\.venv\Scripts\python.exe -m pip install -e ".[dev]"
.\.venv\Scripts\python.exe scripts\reproduce.py verify
```

macOS or Linux:

```
[bash]
./venv/bin/python -m pip install --upgrade pip setuptools
./venv/bin/python -m pip install -e '[dev]'
./venv/bin/python scripts/reproduce.py verify
```

5. Dataset Acquisition

Download `CSV.zip` from the official `CICIoT2023` website after registration. Save it as `data/raw/CICIoT2023_CSV.zip`.

The reference archive has these integrity values:

```
[text]
Bytes: 1,430,268,604
SHA-256: E211E878D2F39226EA3A854683F91AE6175B628807EE96EEECB7A04A36A28DBD
```

Run:

```
[bash]
python scripts/reproduce.py bootstrap
```

The script checks the archive before extracting its 309 CSV files. It also verifies the included standards and downloads the ETSI document from its official source when needed.

6. How CICIoT2023 Is Used

`CICIoT2023` stores numerical traffic observations. A row contains values such as packet rate, protocol indicators, inter-arrival timing, flag counts, average packet size, and time-to-live behavior. The attack label comes from the source folder, not from a binary flag inside the row.

Thirty attack types are mapped into eight operational families:

Family	Meaning in the planning experiment
Benign	Normal traffic in the dataset
BruteForce	Repeated credential or service-access attempts
DDoS	Distributed high-volume service disruption
DoS	Single-source or non-distributed service disruption
Mirai	Mirai-related botnet activity
Recon	Scanning and information-gathering behavior
Spoofing	Identity or address impersonation behavior
Web	Web-facing exploitation behavior

Infinite values become missing values. Missing values are median-imputed using training data. Number and Tot sum are removed from the primary detector because they can encode aggregation-window size rather than attack behavior.

7. Split And Leakage Controls

The primary attack-type-aware protocol separates source files whenever an attack type has enough files. Smaller attack types use deterministic ordered row blocks. Cross-split duplicates are removed.

Random-row and strict whole-file controls are also reported. This makes the effect of split assumptions visible instead of presenting one optimistic number.

Training fits the detector. A calibration partition fits and selects the confidence mapping and threshold. Final test labels are used only for the final evaluation.

8. Detector And Calibration

The primary detector is a class-balanced Random Forest with 140 trees, maximum depth 22, and minimum leaf size 2. Histogram gradient boosting and Extra Trees provide detector-family sensitivity checks.

Raw, sigmoid, and isotonic confidence are compared across five fixed seeds. The selection uses expected calibration error and Brier score on held-out validation data. Isotonic calibration is selected independently in all five runs and then evaluated on the final test rows.

The detector averages macro-F1 near 0.73. Its role is to generate a realistic mixture of correct and incorrect alerts for the safety workflow, not to claim a new state-of-the-art detector.

9. Confidence Routing

The global route uses one threshold for all predicted families. The family-aware route selects a separate threshold for each predicted family. If a family cannot meet the validation rule, it fails closed and sends those predictions to review.

Family-aware routing retains fewer rows for planning but reduces wrong retained decisions by about 30% on average. This is the intended tradeoff: more caution in exchange for fewer confident mistakes reaching the planner.

10. The Fixed 160-Alert Sample

The planning sample contains 20 alerts from each true traffic family. It deliberately covers detector-correct, detector-error, confidence-qualified, and escalated cases. Seventy-five alerts are errors and eighty fall below the selected threshold.

Only the alert and detector prediction are sent to the planner. Hidden true labels remain in the evaluation field for later measurement.

The fixed cases are included in `data/processed/planning_cases_160.jsonl` so readers can inspect or repeat the planning stages without rebuilding the detector first.

11. Standards Retrieval

The corpus contains seven official NIST, ETSI, and IETF documents. Documents are divided into 1,400-character chunks with 180-character overlap. TF-IDF over unigrams and bigrams returns the top three chunks.

The exact sources, versions, hashes, and redistribution approach are recorded in `docs/standards/`.

Three conditions are compared on the same alerts:

Condition	Evidence supplied
Relevant RAG	Retrieval based on the predicted family and observed protocols
No RAG	No evidence context
Wrong-family RAG	Retrieval based on a deterministic different family

The wrong-family condition is a negative control. It tests whether the model treats any valid identifier as meaningful evidence.

12. Planning And Guardrails

GPT-5.4 mini produces compact JSON with a threat assessment, recommended actions, rationales, and evidence identifiers. It does not control the final target, duration, approval, rollback, or execution state.

Deterministic normalization then:

- copies the validated alert scope;
- removes family-incompatible actions;
- replaces uncertain disruption with cautious responses;
- clips durations and parameters to configured limits;
- removes identifiers outside the retrieved set;
- limits the plan to three actions;
- requires approval for disruptive actions;
- adds rollback information;
- sets PROPOSED_NOT_EXECUTED;
- independently rechecks the full schema and policy.

Raw and normalized validity are both reported. The normalized result demonstrates enforcement of declared rules, not model accuracy.

13. Action-Level Evidence Gate

The relevant-RAG plans contain 375 proposed actions. The wrong-family plans contain 376. The totals exceed 160 because a plan can contain multiple actions.

For every action, the gate retrieves again using the predicted family, action, protocols, and observations. It verifies citation membership and family/action compatibility. GPT-5.4 mini labels support, while a separate GPT-5.4 evaluator later judges the before and after action sets without seeing the gate decision.

The strict gate keeps direct support. It keeps general support only for non-disruptive actions. Unsupported actions are removed. If a plan becomes empty, action-weighted retrieval chooses the best-supported cautious fallback.

Relevant-RAG results:

Stage	Supported	Unsupported	Total
Before gate	259	116	375
After gate	193	19	212

Wrong-family recovery results:

Stage	Supported	Unsupported	Total
Before gate	259	117	376
After gate	199	14	213

The gate creates a smaller and cleaner action set. It does not increase the absolute number of supported actions. Its strongest recovery result is that disruptive actions fell from 33 to zero after the wrong-family starting condition.

14. Independent Model Reviews

Reviewer A used ChatGPT with GPT-5.6 Sol Ultra. Reviewer B used Gemini with Gemini Pro Extended. Both independently scored blinded plan and action-evidence records.

Each reviewed 192 plans and 128 evidence items. Both generally rated the final plans as useful and cautious. They differed in rating strictness, especially for direct versus general evidence support. Agreement and condition summaries are preserved in `reports/comprehensive_evaluation/model_audit/`.

These are model-based reviews. Blank packets for an additional security-aware human review are stored separately.

15. Cost Controls

No key is stored in the repository. Set `OPENAI_API_KEY` only in the local process environment.

Each paid stage has its own limit:

Stage	Call ceiling	Estimated-cost ceiling
Expanded planning	500	USD 4.00
GPT-5.4 sensitivity	110	USD 3.00
Relevant-RAG evidence gate	20	USD 2.50
Wrong-family recovery gate	20	USD 2.50

The implementation checks the estimate before starting and saves every successful response immediately. The recorded successful runs cost about USD 4.63 across these four stages.

16. Exact Execution Order

```
[bash]
python scripts/reproduce.py bootstrap
python scripts/reproduce.py offline
python scripts/reproduce.py journal-offline
python scripts/reproduce.py journal-agent
python scripts/reproduce.py model-sensitivity
python scripts/reproduce.py evidence-gate
python scripts/reproduce.py mismatched-evidence-gate
python scripts/reproduce.py model-audit
python scripts/reproduce.py journal-reports
python scripts/reproduce.py verify
```

Paid commands display their limits and request confirmation. Use new run IDs for new scientific conditions. Reusing a reference run ID resumes its completed records.

17. Where To Find The Evidence

Path	Contents
experiments/runs/reference_evaluation/three_condition_planning_160_alerts/	Primary 480 planning outputs
experiments/runs/reference_evaluation/gpt54_model_sensitivity_32_alerts/	Stronger-model outputs
experiments/runs/evidence_gates/relevant_rag_160_alerts/	Relevant-RAG gate records
experiments/runs/evidence_gates/wrong_family_recovery_160_alerts/	Wrong-family recovery records
reports/comprehensive_evaluation/tables/	Derived machine-readable summaries
reports/comprehensive_evaluation/model_audit/	Completed model judgments and agreement
docs/results/RESULT_TO_ARTIFACT_MAP.md	Result-to-code and result-to-file map

18. Verification

Run:

```
[bash]  
python scripts/reproduce.py verify
```

The verification stage checks tests, expected record counts, condition balance, evidence-gate totals, final schema and policy validity, standards hashes, documentation links, excluded publication files, personal absolute paths, and common API-key patterns.

19. Responsible Interpretation

The experiments support confidence-aware routing, source traceability, deterministic conformance, action-level evidence filtering, and cautious recovery from mismatched retrieval.

They do not claim that a citation formally authorizes an action, that the detector is always correct, or that a live attack was stopped. Those questions belong to a future controlled gateway study with site policy, authorization, rollback, and service-impact measurement.

20. Troubleshooting

- If `iot_poc` cannot be imported, run the editable installation command from Section 4.
- If the dataset hash fails, obtain a fresh official archive and do not continue with a partial download.
- If ETSI download fails, retry from a normal network connection and verify the URL in `docs/standards/manifest.json`.
- If a paid run stops, rerun the same command. Completed records are detected and skipped.
- If the preflight estimate exceeds a cap, create a new configuration with a smaller case limit or a deliberately approved ceiling.
- Docker errors do not affect this project because Docker is not part of the experimental pipeline.